



NUST CHIP DESIGN CENTRE

RISC-V Architecture

Lab Manual # 20

RISC-V Datapath Implementation **(Upper Immediate and Branches)**

<u>Name</u>	<i><u>Muddassir Ali Siddiqui</u></i>
<u>Instructor</u>	<i><u>Sir Imran & Sir Umer</u></i>
<u>Date</u>	<i><u>29th Aug 2025</u></i>

1. In-Lab Tasks: (Write your lab task & screenshots here)

i. Given Venus Code:

```
# Test the RISC-V processor:

# add, sub, and, or, slt, addi, lw, sw, beq, jal

# If successful, it should write the value 25 to address 100

#

# RISC-V Assembly      Description      Address  Machine Code
main: addi x2, x0, 5    # x2 = 5          0      00500113
      addi x3, x0, 12   # x3 = 12          4      00c00193
      addi x7, x3, -9   # x7 = (12 - 9) = 3    8      f718393
      or  x4, x7, x2    # x4 = (3 OR 5) = 7    C      0023e233
      and x5, x3, x4    # x5 = (12 AND 7) = 4   10     0041f2b3
      add x5, x5, x4    # x5 = 4 + 7 = 11     14     004282b3
      beq x5, x7, end   # shouldn't be taken  18     02728663
      slt x4, x3, x4    # x4 = (12 < 7) = 0   1C     40720333
      beq x4, x0, around # should be taken    20     00020663
      addi x5, x0, 0    # shouldn't execute   24     00000293
around: slt x4, x7, x2  # x6 = (3 < 5) = 1     28     004323b3
      add x7, x4, x5    # x6 = (1 + 11) = 12   2C     0062a2b3
      sub x7, x7, x2    # x7 = (12 - 5) = 7    30     0023a3b3
      sw  x7, 84(x3)    # [96] = 7           34     0471aa23
      lw  x2, 96(x0)    # x2 = [96] = 7       38     06012083
      add x9, x2, x5    # x9 = (7 + 11) = 18   3C     005104b3
      jal x3, end       # jump to end, x3 = 0x44 40     008000ef
      addi x2, x0, 1    # shouldn't execute   44     00100113
end: add x2, x2, x9     # x2 = (7 + 18) = 25   48     00910133
      sw  x2, 0x20(x3)  # [100] = 25          4C     0621a023
done: beq x2, x2, done  # infinite loop      50     00210063
```

```

1 # Test the RISC-V processor:
2 #   add, sub, and, or, slt, addi, lw, sw, beq, jal
3 # If successful, it should write the value 25 to address 100
4 #
5 #   RISC-V Assembly      Description      Address  Machine Code
6 main: addi x2, x0, 5      # x2 = 5      0      00500113
7      addi x3, x0, 12      # x3 = 12      4      00c00193
8      addi x7, x3, -9      # x7 = (12 - 9) = 3      8      f f718393
9      or x4, x7, x2        # x4 = (3 OR 5) = 7      C      0023e233
10     and x5, x3, x4        # x5 = (12 AND 7) = 4      10     0041f2b3
11     add x5, x5, x4        # x5 = 4 + 7 = 11      14     004282b3
12     beq x5, x7, end       # shouldn't be taken      18     02728663
13     slt x4, x3, x4        # x4 = (12 - 7) = 0      1C     40720333
14     beq x4, x0, around    # should be taken      20     00020663
15     addi x5, x0, 0        # shouldn't execute      24     00000293
16 around: slt x4, x7, x2    # x6 = (3 < 5) = 1      28     004323b3
17     add x7, x4, x5        # x6 = (1 < 11) = 12      2C     0062a2b3
18     sub x7, x7, x2        # x7 = (12 + 5) = 7      30     0023a3b3
19     sw x7, 84(x3)         # [96] = 7      34     0471aa23
20     lw x2, 96(x0)         # x2 = [96] = 7      38     06012083
21     add x9, x2, x5        # x9 = (7 + 11) = 18      3C     005104b3
22     jal x3, end           # jump to end, x3 = 0x44      40     008000ef
23     addi x2, x0, 1        # shouldn't execute      44     00100113
24 end: add x2, x2, x9       # x2 = (7 + 18) = 25      48     00910133
25     sw x2, 0x20(x3)       # [100] = 25      4C     0621a023
26 done: beq x2, x2, done    # infinite loop      50     00210063
27

```

ii. Instruction Memory:

> pc[31:0]	00000050	00000000	00000004	00000008	0000000c	00000010	00000014	00000018	0000001c	00000020	00000028	0000002c	00000030	00000034
> next_pc[31:0]	00000050	00000004	00000008	0000000c	00000010	00000014	00000018	0000001c	00000020	00000028	0000002c	00000030	00000034	00000038
> instr[31:0]	00210063	00500113	00c00193	ff718393	0023e233	0041f2b3	004282b3	02728663	0041a233	00030463	0023a233	005103b3	0023a3b3	0471aa23
> pc[31:0]	00000050	00000038	0000003c	00000040	00000048	0000004c				00000050				
> next_pc[31:0]	00000050	0000003c	00000040	00000048	0000004c					00000050				
> instr[31:0]	00210063	0471aa23	06002103	005104b3	008001ef	00910133	0221a023			00210063				

iii. After 5 iterations:

Venus Output:

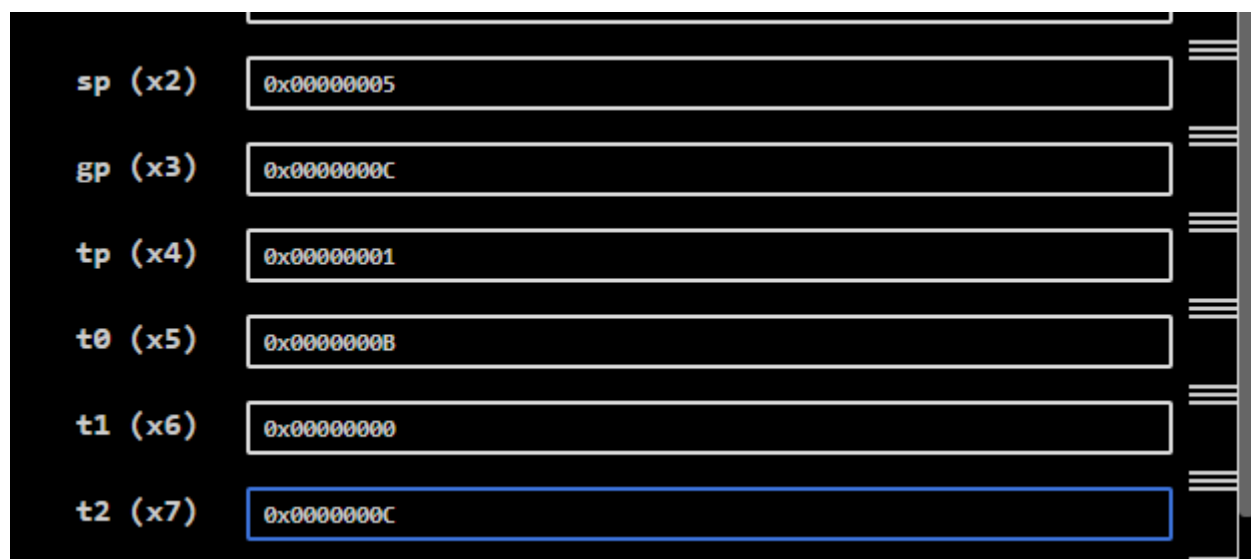
sp (x2)	0x00000005
gp (x3)	0x0000000C
tp (x4)	0x00000007
t0 (x5)	0x00000004
t1 (x6)	0x00000000
t2 (x7)	0x00000003

My RTL Output:

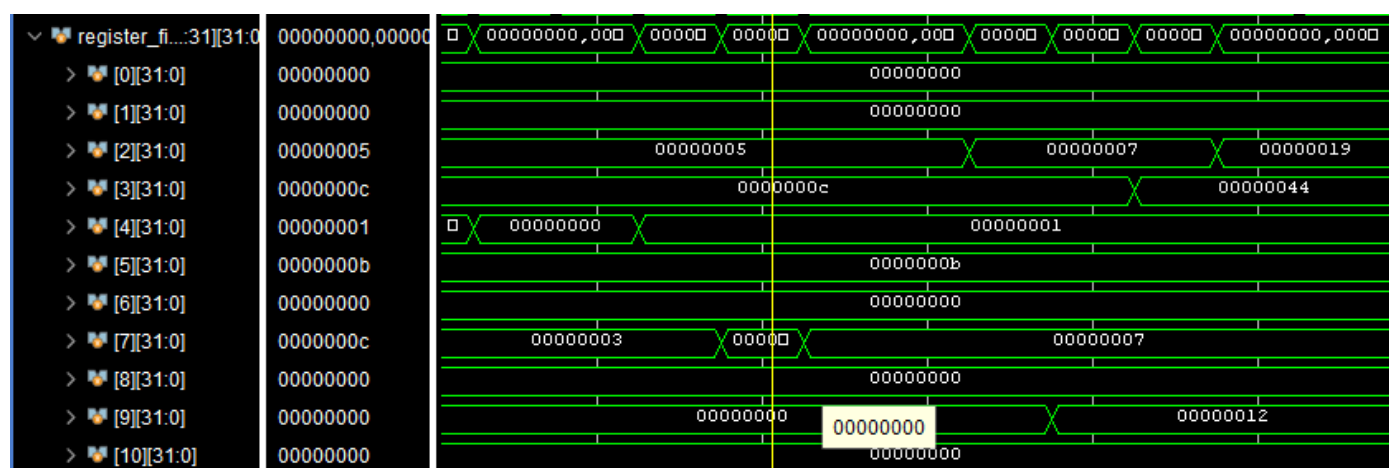
register_file[31:0]	00000000,00000000	00000000	00000000	00000000	00000000	00000000,00000000	00000000,00000000	00000000
> [0][31:0]	00000000					00000000		
> [1][31:0]	00000000					00000000		
> [2][31:0]	00000005	00000000				00000005		
> [3][31:0]	0000000c	00000000				0000000c		
> [4][31:0]	00000007	00000000				00000007	00000000	00000000
> [5][31:0]	00000004	00000000				00000000	0000000b	
> [6][31:0]	00000000					00000000		
> [7][31:0]	00000003	00000000				00000003		
> [8][31:0]	00000000					00000000		
> [9][31:0]	00000000					00000000		
> [10][31:0]	00000000					00000000		
> [11][31:0]	00000000					00000000		
> [12][31:0]	00000000					00000000		

iv. After 12 iterations:

Venus Output:



My RTL Output:



v. After 18 iterations:

Venus Output:

sp (x2)	0x00000007
gp (x3)	0x00000044
tp (x4)	0x00000001
t0 (x5)	0x0000000B
t1 (x6)	0x00000000
t2 (x7)	0x00000007
s0 (x8)	0x00000000
s1 (x9)	0x00000012

My RTL Output:

register_file[31:0]	00000000,00000000	00000000,00000000	00000000,00000000	00000000,00000000	00000000,00000000	00000000,00000000	00000000,00000000
> [0][31:0]	00000000		00000000				
> [1][31:0]	00000000		00000000				
> [2][31:0]	00000007	00000005		00000007		00000019	
> [3][31:0]	00000044	0000000c				00000044	
> [4][31:0]	00000001	00000000		00000001			
> [5][31:0]	0000000b		0000000b				
> [6][31:0]	00000000		00000000				
> [7][31:0]	00000007	00000003	00000000	00000007			
> [8][31:0]	00000000		00000000				
> [9][31:0]	00000012	00000000				00000012	
> [10][31:0]	00000000		00000000				

vi. Final result:

Venus Output:

ra (x1)	0x00000000
sp (x2)	0x00000019
gp (x3)	0x00000044
tp (x4)	0x00000001
t0 (x5)	0x0000000b
t1 (x6)	0x00000000
t2 (x7)	0x00000007
s0 (x8)	0x00000000
s1 (x9)	0x00000012
a0 (x10)	0x00000000

My RTL Output:

> [0][31:0]	00000000
> [1][31:0]	00000000
> [2][31:0]	00000019
> [3][31:0]	00000044
> [4][31:0]	00000001
> [5][31:0]	0000000b
> [6][31:0]	00000000
> [7][31:0]	00000007
> [8][31:0]	00000000
> [9][31:0]	00000012
> [10][31:0]	00000000

vii. All Signals:



This is infinite loop instruction always here exist.

Jump signal is high for jump by jal.

ALU uses operand b to apply operations, so it selects either register file's input or immediate.

ALU uses operand a to apply operations, so it selects either register file's input or pc.

Branch signal is high and if true then jump is also high.

viii. Data Memory:

0x00000064	00	00	00	19
0x00000060	00	00	00	07

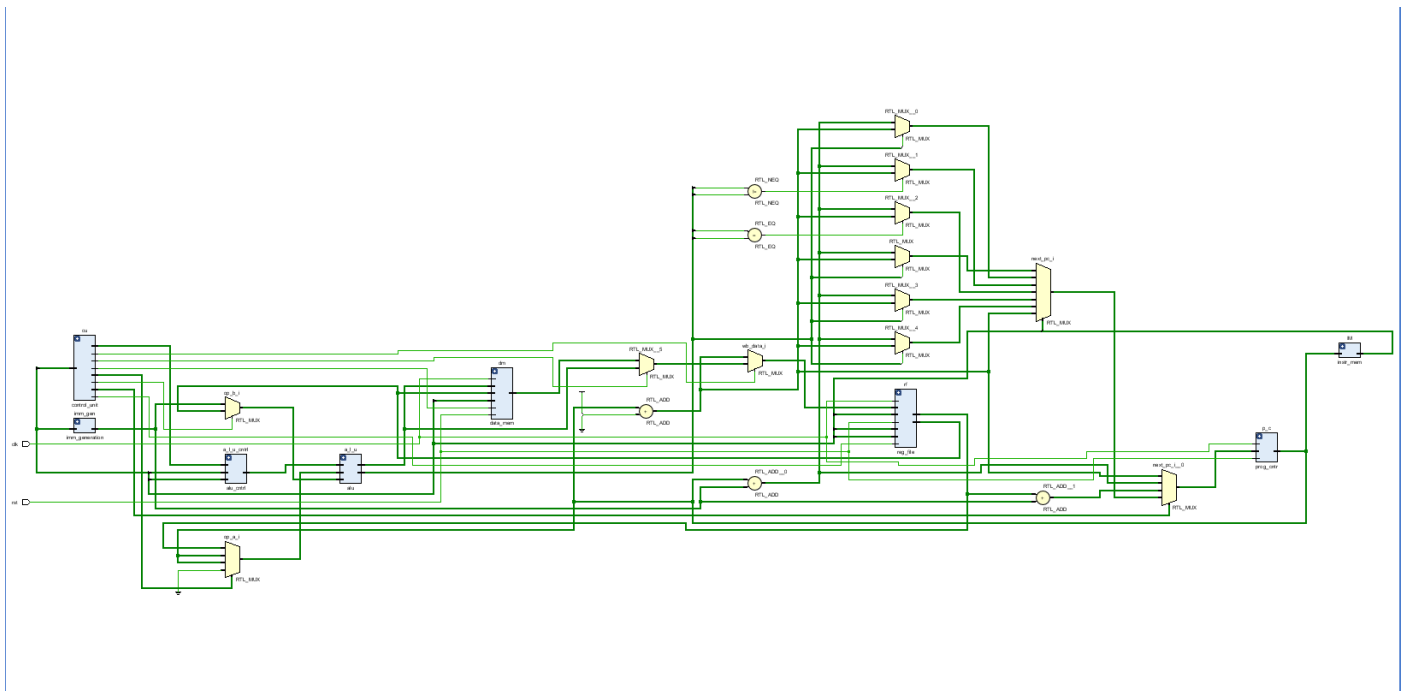
> [24][31:0]	00000007	00000007
> [25][31:0]	00000019	00000019

Ox64 is 96 in decimal but divided by 4 got 24, that's why it stores at location 24.

ix. TCL report:

```
INFO: [USF-XSim-8] Loading simulator feature
Vivado Simulator 2019.1
Time resolution is 1 ps
open_wave_config F:/Fpga_Projects/lab_20/RISBUJ/top_tb_behav.wcfg
source top_tb.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0 } {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#     send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->Ne
#   }
# }
# run 1000ns
WARNING: 0ns : none of the conditions were true for unique case from File:F:/Fpga_Projects/lab_20/RISBUJ/srcs/sources_1/imports/new/data_mem.sv:119
INFO: [USF-XSim-96] XSim completed. Design snapshot 'top_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:03 ; elapsed = 00:00:05 . Memory (MB): peak = 3071.602 ; gain = 0.000
```

x. Schematic:



2. Critical Analysis: (Write your critical analysis / conclusion here)

By designing and simulating the processor, I was able to understand the execution of fundamental instructions such as arithmetic, logic, load/store, and control flow. The lab highlighted the simplicity and modularity of the RV32I design, making it both educational and efficient for hardware realization. Through this exercise, I gained hands-on experience with Datapath, control signals, and instruction decoding, reinforcing theoretical concepts with practical application.

RISC-V Architecture

Overall, this implementation deepened my understanding of processor design and laid a strong foundation for exploring more advanced RISC-V extensions.

In conclusion, the implementation of the RISC-V RV32I base architecture provided practical insight into how an instruction set architecture (ISA) operates at the hardware level.